

Efficient Filtering and Fitting of Models Derived from Integro-Difference Equations

Evan Tate Paterson Hughes

Table of contents

Chapter 1

Introduction

The Integro-Difference equation model (here abbreviated as IDEM ¹) is a dynamics-based spatio-temporal aiming to model diffusion and convection by making the value of a process a weighted average of it's previous time, plus noise.

As common and widespread as the problem is, spatio-temporal modelling still presents a great deal of difficulty. Inherently, spatio-temporal datasets are almost always high-dimensional, especially when considering image data (where each pixel might be considered an observation). Additionally, repeated observations are usually not possible due to their time dependence.

Traditionally, the problem has been tackled by the moments (usually the means and covariances) of the process in order to make inference (Wikle, Zammit-Mangion, and Cressie (2019), for example, call this ‘descriptive’ modelling). While this method can be sufficient for many problems, there are many cases where we are underutilizing some knowledge of the underlying dynamic systems involved. For instance, in temperature models, we know that temperature has movement (convection) and spread (diffusion), and that the state at any given time will depend on its state at previous times ². We call models which make use of this ‘dynamic’ models.

¹Historically, this has been abbreviated as IDE. However, with that abbreviation almost universally meaning ‘Integrated Development Environment’, here, we choose to include the ‘M’ in the abbreviation.

²at least, in a discrete-time scenario. Integro-difference based mechanics can be derived from continuous-time convection-diffusion processes, see Liu, Yeo, and Lu (2022)

Chapter 2

Integro-difference Based Dynamics

A general way of writing a hierarchical (discrete-time) dynamical model might be

$$\begin{aligned} Y_{t+1}(\cdot) &= \mathcal{M}_t(Y_0(\cdot), \dots, Y_t(\cdot)) + \omega_t(\cdot), \quad t = 0, \dots, T-1, \\ Z_t(\cdot) &= \mathcal{O}_t(Y_t(\cdot)) + x(\cdot)^\top \beta + \epsilon_t(\cdot), \quad t = 1, \dots, T. \end{aligned}$$

This describes the scalar random fields $Z_t(\cdot), Y_t(\cdot) \in \mathbb{R}$ over the space $\mathcal{D} \subset \mathcal{R}^d$, which are the observed data and unobserved dynamic process, respectively. $\mathcal{M}_t$ here is a non-random ‘propagation operator’, defining how the process evolves with respect to its previous state(s), and $\mathcal{O}_t$ is a non-random ‘observation operator’, defining how observations of a given process state are taken. Both these fields have random (usually time-independent) additive random fields, $\omega_t(\cdot), \epsilon_t(\cdot)$, and we also include non-random measured linear covariate terms $x(\cdot)^\top \beta$.

If we discretize the space into n spatial locations $\{\mathbf{s}_i\}_{i=1, \dots, n}$, assume the operators are linear, assert a Markov condition, and assume the errors are all normal, we get a simple linear dynamic system;

$$\begin{aligned} \mathbf{Y}_{t+1} &= M_t \mathbf{Y}_t + \boldsymbol{\omega}_t, \quad t = 0, \dots, T-1, \\ \tilde{\mathbf{Z}}_t &= O_t \mathbf{Y}_t + \boldsymbol{\epsilon}_t, \quad t = 1, \dots, T, \end{aligned} \tag{2.1}$$

where we have written $\mathbf{Y}_t = (Y_t(\mathbf{s}_1), \dots, Y_t(\mathbf{s}_n))$, and similar for $\mathbf{Z}_t, \boldsymbol{\epsilon}_t$ and $\boldsymbol{\omega}_t$, and we have written $\tilde{\mathbf{Z}}_t = \mathbf{Z}_t + X^\top \beta$. This is a well-known type of system, the process Y can easily be estimated either directly or with a Kalman filter/smoothing and variants, which will be discussed later.

However, this model is restrictive and high-dimensional; M_t , the primary quantities which need estimation, is of dimension $n \times n$, of which there are T matrices to be estimated. Even if we allow the propagation matrix to be invariant in time, we can still only make predictions at the stations $\{\mathbf{s}_i\}$.

This motivates a different approach; in particular, one which allows us to estimate the random field at arbitrary points $Y_t(\mathbf{s})$ using some spectral decomposition, which would alleviate these problems.

The Integro-difference equation model attempts to generalise Equation ?? into the continuous space by replacing the discrete linear M_t by a continuous integral equivalent;

$$\begin{aligned} Y_{t+1}(\mathbf{s}) &= \int_{\mathcal{D}_s} \kappa_t(\mathbf{s}, \mathbf{r}) Y_t(\mathbf{r}) d\mathbf{r} + \omega_t(\mathbf{s}), \quad t = 0, \dots, T-1, \\ Z_t(\mathbf{s}) &= Y_t(\mathbf{s}) + X(\mathbf{s})^\top \beta + \epsilon_t(\mathbf{s}), \quad t = 1, \dots, T. \end{aligned} \tag{2.2}$$

Where $\omega_t(\mathbf{s})$ is a small scale gaussian variation with no temporal dynamics (Cressie and Wikle 2015 call this a ‘spatially descriptive’ component), $\mathbf{X}(\mathbf{s})$ are spatially varying covariates (for example, in a large-scale climate scenario, this might be latitude, concentration of some chemical/element like nitrogen) $\kappa(\mathbf{s}, \mathbf{r})$ is the driving ‘kernel’ function, and ϵ_t is a Gaussian white noise ‘measurement error’ term.

Our operator is now $\mathcal{M}(Y_t(\mathbf{s})) = \int_{\mathcal{D}_s} \kappa_t(\mathbf{s}, \mathbf{r}) Y_t(\mathbf{r}) d\mathbf{r}$, which can model diffusion and convection by choosing the shape of κ (which, from now on, we will assume to be temporally invariant). This kernel defines how each point in space is affected by every other point in space at the previous time. For example, if we choose a Gaussian-like shape,

$$\kappa(\mathbf{s}, \mathbf{r}; \mathbf{m}, a, b) = a \exp\left(-\frac{1}{b}|\mathbf{s} - \mathbf{r} + \mathbf{m}(\mathbf{s})|^2\right),$$

then the ‘flow’ would be in the direction of $-\mathbf{m}(\mathbf{s})$, and the diffusion would be controlled by b and a . This creates a ‘spatially variant kernel’, where the direction of flow varies across the space, as illustrated in Figure ??.

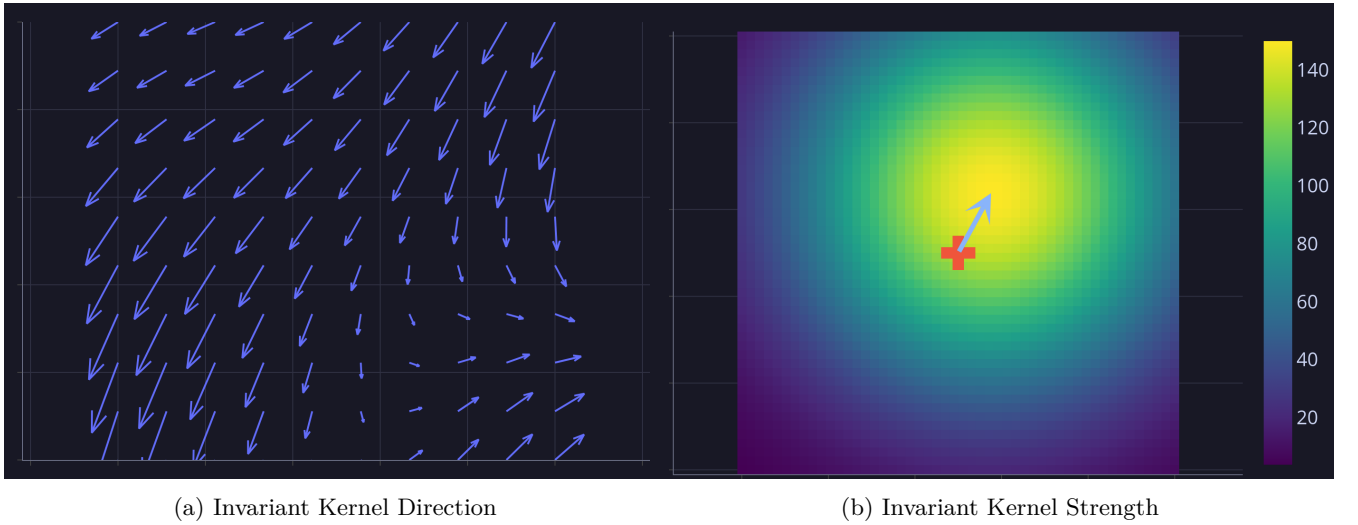


Figure 2.1: A spatially variant kernel across the region $[0, 1] \times [0, 1]$. The kernel direction is shown on the left, and on the right is the amount that each point affects the point $(0.5, 0.5)$, marked with a red cross. ‘Flow’ is allowed to vary by a function $\mathbf{m}(\mathbf{s})$ which is chosen randomly using a basis expansion (see Section ??). The other two parameters are set at $a = 150, b = 0.2$.

Chapter 3

Spectral Representations

The key to being able to computationally work with IDEMs, as perhaps originally made by Wikle and Cressie (1999), is to work with the spectral decomposition of the process, in order to coerce the model hierarchy into a more familiar linear dynamical system form, like Equation ??.

This kind of dimension-reduction allows us to parametrise spatial fields with as few or as many parameters as we want.

3.1 Process decomposition

Choose a complete class of spatial spectral basis functions, $\{\phi_i(\cdot) : \mathcal{D} \rightarrow \mathbb{R}\}_{i=1,\dots,r}$, and decompose the process spatial field at each time;

$$Y_t(\mathbf{s}) \approx \sum_{i=1}^r \alpha_{i,t} \phi_i(\mathbf{s}), \quad t = 0, \dots, T. \quad (3.1)$$

where we truncate the expansion at some $r \in \mathbb{N}$. Notice that we can write this in vector/matrix form, where we consider the vector field $\boldsymbol{\phi}(\cdot) = (\phi_1(\cdot), \dots, \phi_r(\cdot))^\top$; considering times $t = 1, 2, \dots, T$, we set

$$\begin{aligned} \boldsymbol{\phi}(\mathbf{s}) &= (\phi_1(\mathbf{s}), \phi_2(\mathbf{s}), \dots, \phi_r(\mathbf{s}))^\top, \\ \boldsymbol{\alpha}_t &= (\alpha_{1,t}, \alpha_{2,t}, \dots, \alpha_{r,t})^\top. \end{aligned} \quad (3.2)$$

Now, (Equation ??) gives us, for any $\mathbf{s} \in \mathcal{D}$,

$$Y(\mathbf{s}; t) \approx \boldsymbol{\phi}^\top(\mathbf{s}) \boldsymbol{\alpha}(t). \quad (3.3)$$

We can effectively now work exclusively with $\boldsymbol{\alpha}_t = (\alpha_{1,t}, \dots, \alpha_{r,t})^\top$. To do so, we need to find the evolution equation of $\boldsymbol{\alpha}_t$, as given below.

Theorem 3.1.1 (Spectral form of the state evolution). *Define the Gram matrix;*

$$\Psi := \int_{\mathcal{D}_s} \boldsymbol{\phi}(\mathbf{s}) \boldsymbol{\phi}(\mathbf{s})^\top d\mathbf{s}. \quad (3.4)$$

Then, the basis coefficients evolve by the equation

$$\boldsymbol{\alpha}_{t+1} = M\boldsymbol{\alpha}_t + \boldsymbol{\eta}_t, \quad (3.5)$$

where $M = \Psi^{-1} \int \int \boldsymbol{\phi}(\mathbf{s})\kappa(\mathbf{s}, \mathbf{r})\boldsymbol{\phi}(\mathbf{r})^\top d\mathbf{r}d\mathbf{s}$ and $\boldsymbol{\eta}_t = \Psi^{-1} \int \boldsymbol{\phi}(\mathbf{s})\omega_t(\mathbf{s})d\mathbf{s}$.

Proof. (Adapting from Dewar, Scerri, and Kadirkamanathan 2008), write out the process equation, (Equation ??), using the first equation of (Equation ??);

$$Y_{t+1}(\mathbf{s}) = \boldsymbol{\phi}(\mathbf{s})^\top \boldsymbol{\alpha}_{t+1} = \int_{\mathcal{D}_s} \kappa(\mathbf{s}, \mathbf{r})\boldsymbol{\phi}(\mathbf{r})^\top \boldsymbol{\alpha}_t d\mathbf{r} + \omega_t(\mathbf{s}),$$

We then multiply both sides by $\boldsymbol{\phi}(\mathbf{s})$ and integrate over $\mathbf{s}$

$$\begin{aligned} \int_{\mathcal{D}_s} \boldsymbol{\phi}(\mathbf{s})\boldsymbol{\phi}(\mathbf{s})^\top d\mathbf{s} \boldsymbol{\alpha}_{t+1} &= \int \boldsymbol{\phi}(\mathbf{s}) \int \kappa(\mathbf{s}, \mathbf{r})\boldsymbol{\phi}(\mathbf{r})^\top d\mathbf{r}d\mathbf{s} \boldsymbol{\alpha}_t + \int \boldsymbol{\phi}(\mathbf{s})\omega_t(\mathbf{s})d\mathbf{s} \\ \Psi \boldsymbol{\alpha}_{t+1} &= \int \int \boldsymbol{\phi}(\mathbf{s})\kappa(\mathbf{s}, \mathbf{r})\boldsymbol{\phi}(\mathbf{r})^\top d\mathbf{r}d\mathbf{s} \boldsymbol{\alpha}_t + \int \boldsymbol{\phi}(\mathbf{s})\omega_t(\mathbf{s})d\mathbf{s}. \end{aligned}$$

So, finally, pre-multiplying by the inverse of the gram matrix, Ψ^{-1} (Equation ??), we arrive at the result. $\square \quad \square$

3.2 Spectral form of the Process Noise

We still have to set out what the process noise, $\omega_t(\mathbf{s})$, and its spectral counterpart, $\boldsymbol{\eta}_t$, are. Dewar, Scerri, and Kadirkamanathan (2008) fix the variance of $\omega_t(\mathbf{s})$ to be uniform and uncorrelated across space and time, with $\omega_t(\mathbf{s}) \sim \mathcal{N}(0, \sigma^2)$. It is then easily shown that $\boldsymbol{\eta}_t$ is also normal, with $\boldsymbol{\eta}_t \sim \mathcal{N}(0, \sigma^2 \Psi^{-1})$.

However, in practice, we simulate in the spectral domain; that is, if we want to keep things simple, it would make sense to specify (and fit) the distribution of $\boldsymbol{\eta}_t$, and compute the variance of $\omega_t(\mathbf{s})$ if needed.

Lemma 3.2.1. *Let $\boldsymbol{\eta}_t \sim \mathcal{N}(0, \Sigma_\eta)$, and $\text{Cov}[\boldsymbol{\eta}_t, \boldsymbol{\eta}_{t+\tau}] = 0, \forall \tau > 0$. Then $\omega_t(\mathbf{s})$ has covariance*

$$\text{Cov}[\omega_t(\mathbf{s}), \omega_{t+\tau}(\mathbf{r})] = \begin{cases} \boldsymbol{\phi}(\mathbf{s})^\top \Sigma_\eta \boldsymbol{\phi}(\mathbf{r}) & \text{if } \tau = 0 \\ 0 & \text{else} \end{cases}$$

Proof. Consider $\Psi \boldsymbol{\eta}_t$, and consider the case $\tau = 0$. It is clearly normal, with zero expectation and variance (using Equation ??),

$$\begin{aligned} \text{Var}[\Psi \boldsymbol{\eta}_t] &= \Psi \text{Var}[\boldsymbol{\eta}_t] \Psi^\top = \Psi \Sigma_\eta \Psi^\top, \\ &= \int_{\mathcal{D}_s} \boldsymbol{\phi}(\mathbf{s})\boldsymbol{\phi}(\mathbf{s})^\top d\mathbf{s} \Sigma_\eta \int_{\mathcal{D}_s} \boldsymbol{\phi}(\mathbf{r})\boldsymbol{\phi}(\mathbf{r})^\top d\mathbf{r} \\ &= \int \int_{\mathcal{D}_s^2} \boldsymbol{\phi}(\mathbf{s})\boldsymbol{\phi}(\mathbf{s})^\top \Sigma_\eta \boldsymbol{\phi}(\mathbf{r})\boldsymbol{\phi}(\mathbf{r})^\top d\mathbf{r}d\mathbf{s} \end{aligned} \quad (3.6)$$

Since it has zero expectation, we also have

$$\begin{aligned}
\text{Var}[\Psi\boldsymbol{\eta}_t] &= \mathbb{E}[(\Psi\boldsymbol{\eta}_t)(\Psi\boldsymbol{\eta}_t)^\top] = \mathbb{E}[\Psi\boldsymbol{\eta}_t\boldsymbol{\eta}_t^\top\Psi^\top] \\
&= \mathbb{E}\left[\int_{\mathcal{D}_s}\boldsymbol{\phi}(\mathbf{s})\omega_t(\mathbf{s})d\mathbf{s}\int_{\mathcal{D}_s}\boldsymbol{\phi}(\mathbf{r})^\top\omega_t(\mathbf{r})d\mathbf{r}\right] \\
&= \int\int_{\mathcal{D}_s^2}\boldsymbol{\phi}(\mathbf{s})\mathbb{E}[\omega_t(\mathbf{s})\omega_t(\mathbf{r})]\boldsymbol{\phi}(\mathbf{r})^\top d\mathbf{s}d\mathbf{r}.
\end{aligned} \tag{3.7}$$

We can see that, comparing (Equation ??) and (Equation ??), we have

$$\text{Cov}[\omega_t(\mathbf{s}), \omega_t(\mathbf{r})] = \mathbb{E}[\omega_t(\mathbf{s})\omega_t(\mathbf{r})] = \boldsymbol{\phi}(\mathbf{s})^\top \Sigma_\eta \boldsymbol{\phi}(\mathbf{r}).$$

Since, once again, $\mathbb{E}[\boldsymbol{\omega}_t(\mathbf{s})] = 0$.

For the $\tau \neq 0$ case, it is simple to show that the covariance is 0.

□

□

3.3 Kernel Parameterisations

Next is the part of the system, which defines the dynamics; the kernel function, κ . There are a few ways to handle the kernel. One of the most obvious is to expand it out into a spectral decomposition as well;

$$\kappa \approx \sum_i \beta_i \psi(\mathbf{s}, \mathbf{r}).$$

This can allow for a wide range of interestingly shaped kernel functions, but see how these basis functions must now act on $\mathbb{R}^2 \times \mathbb{R}^2$; to get a wide enough space of possible functions, we would likely need many terms in the spectral expansion.

A much simpler approach would be to simply parametrise the kernel function, to $\kappa(\mathbf{s}, \mathbf{r}, \boldsymbol{\theta}_\kappa)$. We then establish a simple shape for the kernel (e.g. Gaussian) and rely on very few parameters (for example, scale, shape, offsets). The example kernel used in the `jaxidem` is a Gaussian-shape kernel;

$$\kappa(\mathbf{s}, \mathbf{r}; \mathbf{m}, a, b) = a \exp\left(-\frac{1}{b}|\mathbf{s} - \mathbf{r} + \mathbf{m}|^2\right).$$

Of course, this kernel lacks spatial dependence. We can add spatial variance back by adding dependence on $\mathbf{s}$ to the parameters, for example, varying the offset term as $\mathbf{m}(\mathbf{s})$. Of course, now we are back to having entire functions as parameters, but taking the spectral decomposition of the parameters we actually want to be spatially variant seems like a reasonable middle ground (Cressie and Wikle 2015). The actual parameters of such a spatially-variant kernel are then the spectral coefficients for the expansion of any spatially variant parameters, as well as any constant parameters. This is precisely what is plotting in Figure ??, where the spectral coefficients are randomly sampled from a multivariate normal distribution;

$$\mathbf{m}(\mathbf{s}) = \begin{pmatrix} \sum_{i=1}^{r_m} \phi_{\kappa,i}(\mathbf{s}) m_i^{(x)} \\ \sum_{i=1}^{r_m} \phi_{\kappa,i}(\mathbf{s}) m_i^{(y)} \end{pmatrix},$$

where $m_i^{(x)}$ and $m_i^{(y)}$ are coefficients for the x and y coordinates respectively, and $\phi_{\kappa,i}(\mathbf{s})$ are basis functions (e.g. bisquare¹) functions in Figure ??).

3.4 IDEM as a linear dynamical system

To summarise, we have taken a truncated spectral decomposition to write the Integro-difference equation model as a more traditional linear dynamical system form (Equation ??). All that is left is to include our observations in our system.

Lets assume that at each time t there are n_t observations at locations $\mathbf{s}_{1,t}, \dots, \mathbf{s}_{n_t,t}$. We write the vector of the process at these points as $\mathbf{Y}(t) = (Y(s_{1,t};t), \dots, Y(s_{n_t,t};t))^\top$, and, in it's expanded form $\mathbf{Y}_t = \Phi_t \boldsymbol{\alpha}_t$, where $\Phi \in \mathbb{R}^{r \times n_t}$ is

$$\{\Phi_t\}_{i,j} = \phi_i(s_{j,t}).$$

For the covariates, we write the matrix $X_t = (\mathbf{X}(\mathbf{s}_{1,t}), \dots, \mathbf{X}(\mathbf{s}_{1=n_t,t}))^\top$. We then have

$$\begin{aligned} \mathbf{Z}_t &= \Phi_t \boldsymbol{\alpha}_t + X_t \boldsymbol{\beta} + \boldsymbol{\epsilon}_t, \quad t = 1, \dots, T, \\ \boldsymbol{\alpha}_{t+1} &= M \boldsymbol{\alpha}_t + \boldsymbol{\eta}_t, \quad t = 0, 1, \dots, T-1, \\ M &= \int_{\mathcal{D}_s} \phi(\mathbf{s}) \phi(\mathbf{s})^\top d\mathbf{s} \int_{\mathcal{D}_s^2} \phi(\mathbf{s}) \kappa(\mathbf{s}, \mathbf{r}; \boldsymbol{\theta}_\kappa) \phi(\mathbf{r})^\top d\mathbf{r} d\mathbf{s}, \end{aligned}$$

Writing $\tilde{\mathbf{Z}}_t = \mathbf{Z}_t - X_t \boldsymbol{\beta}$,

$$\begin{aligned} \tilde{\mathbf{Z}}_t &= \Phi_t \boldsymbol{\alpha}_t + \boldsymbol{\epsilon}_t, \quad t = 1, 2, \dots, T, \\ \boldsymbol{\alpha}_{t+1} &= M \boldsymbol{\alpha}_t + \boldsymbol{\eta}_t, \quad t = 0, 1, \dots, T-1. \end{aligned} \tag{3.8}$$

We should also initialise $\boldsymbol{\alpha}_0 \sim \mathcal{N}^r(\mathbf{m}_0, \Sigma_0)$, and fix simple distributions to the noise terms,

$$\begin{aligned} \boldsymbol{\epsilon}_t &\stackrel{\text{iid}}{\sim} \mathcal{N}_{n_{\text{obs}}}(0, \Sigma_\epsilon), \\ \boldsymbol{\eta}_t &\stackrel{\text{iid}}{\sim} \mathcal{N}_R(0, \Sigma_\eta), \end{aligned}$$

which are independent in time.

As in, for example, (Wikle and Cressie 1999), Equation ?? is now in a traditional enough form that the Kalman filter can be applied to filter and compute many necessary quantities for inference, including the marginal likelihood. We can use these quantities in either an EM algorithm or a Bayesian approach, or directly maximise the marginal data likelihood

We now move on to an example simulation of this kind of model using its spectral decomposition and `jaxidem`.

¹The bisquare functions, here, $\phi_i(\mathbf{s}) = [1 - \frac{\|\mathbf{s} - \mathbf{c}_i\|}{w_i}]^2 \cdot I(\|\mathbf{s} - \mathbf{c}_i\| < w_i)$ for i ‘centroids’ or ‘knots’, $\mathbf{c}_i \in \mathcal{D}$, each with ‘radius’ w_i

3.5 Example Simulation

We can now use the above to simulate easily from such models; once we have chosen the appropriate decompositions, we simply compute M and propagate α_t as we would when simulating any other linear dynamic system. We then use the spectral coefficients to generate $Y_t(s)$ and $Z_t(s)$ in the obvious way.

`jaxidem` implements this in the function `sim_idem`, or through the more user-friendly method `idem.IDEM.simulate`. An object of the `IDEM` class contains all the necessary information about basis decompositions, and the `simulate` method calls `simIDEM` without compromising its jit-ability (although just-in-time computation obviously isn't as important for simulation, the jit-ed function could save compile time if someone want to simulate from many models).

The `gen_example_idem` method creates a simple `IDEM` object without many required parameters;

```
key = jax.random.PRNGKey(42)
keys = jax.random.split(key, 3)

model = idem.gen_example_idem(keys[0], k_spat_inv=False)
# Simulation
T = 35
nobs = 50

coords = jax.random.uniform(
    keys[1],
    shape=(nobs, 2),
    minval=0,
    maxval=1,
)

times = jnp.repeat(jnp.arange(1, T + 1), coords.shape[0])
rep_coords = jnp.tile(coords, (T, 1))
x = rep_coords[:,0]
y = rep_coords[:,1]

process_data, obs_data = model.simulate(keys[2], x, y, times)
```

The resulting objects are of class `st_data`, containing a couple of niceties for handling spatio-temporal data, while still storing all data as JAX arrays.

The optional `plotters` module contains some functions to produce plotly plots of data in this format;